A successful cast can hide invalid input

10 October 2026 | C003a deliberate failure | Synthetic teaching examples

Sidiq review status: Not yet reviewed. Original study: October 10; integrated October 11.

Research question and hypothesis

Can converting fractional text to a floating-point number and then to int silently send the wrong horizon into our whole-year savings calculation? Hypothesis: the existing parser rejects 2.5 before calculation, while a deliberately wrong adapter returns 2 and produces a plausible two-year balance. A rejection test must fail when that adapter replaces the parser.

This completes the failure lesson originally dated October 9, on October 10. Yesterday's independent boundary oracle established 75 exact agreements. Today adds one bounded failure scenario with two valid controls and three forbidden spellings. Reproduction and consolidation remain separate lessons.

Beginner explanation and terms

Text preserves what was supplied: the three characters 2, dot, 5. A double stores a numerical value. An int stores a whole number. A cast asks C++ to convert between types; it does not certify that the original input follows our model's rules. Truncation removes the fractional part toward zero.

Our grammar accepts nonempty ASCII decimal digits, with a value from zero through the target int maximum. Leading zeros are allowed. A dot is forbidden even in 2.0: its numerical value is whole, but its spelling violates the agreed interface. Rejection means no returned years and no savings calculation. It must not be replaced with the legitimate value zero.

Mathematics and worked example

For these small nonnegative numbers, T(x) discards the fraction: T(2.5) = 2 and T(0.5) = 0. T(2.0) also equals 2. Multiple inputs therefore produce the same integer. After this information loss, the savings function cannot tell whether 2 came from valid 002 or forbidden 2.5.

The whole-year model is B(n) = P * (1+r)^n. P is principal in money units, r is the effective annual rate as a fraction, and n is a nonnegative whole count of model years. With P = 1000 and r = 0.05: B(0) = 1000; B(1) = 1000 + 50 = 1050; B(2) = 1050 + 52.50 = 1102.50. The wrong adapter reports 1102.50 for text 2.5. This is a correct answer to an unrequested two-year question. We have not defined a fractional-year balance or a financial loss from that substitution.

Original text	Existing parser and pipeline	Wrong adapter and pipeline
002	2 years; 1102.50	2 years; 1102.50
0	0 years; 1000.00	0 years; 1000.00
2.5	Rejected; no calculation	2 years; 1102.50
0.5	Rejected; no calculation	0 years; 1000.00
2.0	Rejected; no calculation	2 years; 1102.50

Implementation, line by line

The fixed experiment is fractional_failure.cpp. The existing years.cpp and savings.cpp are linked without edits. Two builds select either the existing parser or the deliberately wrong adapter. WRONG_ADAPTER is a compile-time test switch, not an external savings interface.

1. std::size_t consumed = 0 creates a counter. std::stod(text, &consumed) converts text to a double and writes the first unconverted position into that counter. The ampersand supplies its address. For 2.5, consumed equals the string length: full consumption alone does not enforce integer grammar.

2. The wrong adapter rejects incomplete conversion, nonfinite values and values outside 0 through 10. This deliberately narrow demonstration bound keeps every cast defined. It is not a proposed replacement for the full parser or a general range-validation technique.

3. return static_cast<int>(value) performs the information-losing conversion. For the fixed 2.5 fixture it returns 2. The explicit syntax makes the cast visible; it does not make it valid policy.

4. The correct build calls parse_whole_years(text). Its character check rejects a dot before returning an integer. examine catches only parsing invalid_argument, prints rejection and returns. The savings call is after that catch, so a financial error cannot masquerade as a parser rejection.

5. On acceptance, compound_balance receives the parsed integer. Each disagreement with the required accept/reject decision increments failures. The program returns exit code 1 when any mismatch exists. This is intentional failure reporting, not a compiler error or a damaged production implementation.

Evaluation protocol and observed checks

Inputs are five fixed synthetic strings, with no historical period or market data. Expected acceptance and output bytes were written from the contract. make cpp-fractional-failure builds both variants with C++17 warnings treated as errors. The Python verifier runs each twice and checks output bytes, diagnostics and exit status exactly. Python Decimal independently verifies the zero-year and two-year arithmetic. No tolerance is needed for these two-decimal displayed outputs; this does not establish exact-money arithmetic generally.

Check	Observed result
Correct pipeline	5 cases pass: 2 accepted; 3 rejected before calculation
Deliberately wrong adapter	3 contract mismatches; required failure exit 1
Replay and independent arithmetic	Both builds repeat exactly; Decimal gives 1102.50
Existing make check	608 Python tests; 66 savings and 30 parser checks pass
Existing oracle and output fixtures	75 comparisons and 4 golden outputs pass
Quality and dependencies	Ruff, 51 formatted files, mypy on 21 files and pip check pass

October 10 checks passed; catch scope was tightened and retested. October 11 integration freshly compiled all nine executables warning-clean under Apple Clang 21. Production code is unchanged.

Reproduce

From the repository root, run make cpp-fractional-failure, make check and .venv/bin/python -m pip --no-cache-dir check. The verifier expects the wrong executable to fail. Its own success means both the correct behaviour and the intended failure were confirmed.

Assumptions, costs and limitations

Savings assumes a fixed effective annual rate, reinvestment, no interim cash flows and no fees, taxes or inflation. No spread, commission, turnover or market profit is measured. Omitted costs can reduce real wealth. Fractional periods would require a separately agreed accrual convention.

The wrong adapter is safe only within its small demonstration bound. General floating-to-integer conversion requires representability; nonfinite or excessive values are not safe casting inputs. Five cases do not establish exhaustive correctness, security, coverage or a mutation score. The existing parser still has no input-length cap. No empirical access gate or personal review is waived.

Primary sources consulted on 10 October

Microsoft's Standard conversions reference explains truncation toward zero and the representability restriction for floating-to-integer conversion. It supports the language behaviour, not our grammar. https://learn.microsoft.com/en-us/cpp/cpp/standard-conversions?view=msvc-170

Microsoft's string functions reference describes stod and its unconverted-position output. It supports the demonstration that floating conversion can consume 2.5 fully. Digit-only acceptance is the project's deliberate interface policy, not a requirement imposed by this library function. https://learn.microsoft.com/en-us/cpp/standard-library/string-functions?view=msvc-170#stod

Interview explanation

Validate original text before discarding information. I demonstrated that a fully consumed floating conversion followed by a cast can still violate a whole-year grammar. The correct pipeline rejected three forbidden strings; a deliberately wrong build accepted them and the harness failed as intended. A plausible financial output cannot validate the input that produced it.

Progress, cadence and next task

October 10's draw was 0, so the completed lesson stayed in the ignored pending area. On October 11, all original input and artifact hashes matched. Build integration, exact replays and full checks were repeated; this note keeps its original study date. The shared October 11 draw is 1 and this coherent integration uses that slot. Commit and remote observations are recorded separately.

Next is C003a reproduction and teach-back, then consolidation. Do not repeat this failure lesson merely because its publication is pending. C003a/C003 remain in progress; C003b, vectors/statistics and the ledger remain conditional. Q004a lawful access and Q005b integration still gate empirical research. No input blocks the next synthetic lesson. Personal review remains Not yet reviewed.

Check your understanding

1. Why can the savings function not reject the original 2.5 after it receives integer 2?

2. Why does full consumption by stod fail to establish our whole-year grammar?

3. Why reject 2.0 even though its numerical value is a whole number?

4. What does the wrong build's exit code 1 establish, and what does it not establish?

Suggested answers

1. The cast discarded the fraction. The function sees the same integer as a valid two-year input; it cannot reconstruct the original spelling. Preserve and validate text before conversion.

2. The floating grammar permits a decimal point. Consuming all three characters only establishes that the floating converter used all the input; our stricter digit-only rule is a separate check.

3. The agreed interface admits digits only. A different policy could allow decimal spellings, but changing that policy requires an explicit contract change, not silent normalization.

4. The harness detected three forbidden acceptances in this fixed negative control. It does not prove all inputs are safe, measure test coverage, or establish investment performance or mastery.